

PHASE 01 · MYSQL QUERY OPTIMIZATION

MySQL은 어떻게 덜 읽는가?

실행 계획부터 커버링 인덱스까지

DB 면접에서 근거를 말하는 5주 학습

다섯 주차는 하나의 최적화 흐름이다

목표는 인덱스를 많이 만드는 것이 아니다

실행 계획을 읽고 검색·정렬·집계·테이블 접근 비용을 단계적으로 줄인다.

- ① 1주차 · 실행 경로 확인
- ② 2주차 · 검색 범위 축소
- ③ 3주차 · 정렬·조인 비용 축소
- ④ 4주차 · 집계 입력 축소
- ⑤ 5주차 · 테이블 접근 생략

MySQL 엔진은 계획하고 InnoDB는 읽는다

MySQL 엔진

연결·파싱·검증·최적화·실행을 담당

InnoDB

페이지·인덱스·레코드 I/O를 담당

면접 한 문장

옵티마이저가 경로를 고르고 스토리지 엔진이 실제 데이터를 읽습니다.

4.1.1 MySQL의 전체 구조

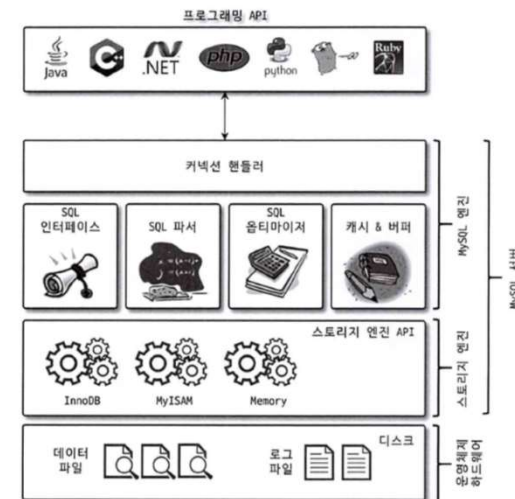


그림 4.1 MySQL 서버의 전체 구조

SQL은 선언이고 실행 방법은 옵티마이저가 정한다

Parser · 문법과 파스 트리

Preprocessor · 객체·권한 검증

Optimizer · 후보 계획의 비용 비교

Execution Engine · 선택한 계획 실행

InnoDB · 페이지와 레코드 읽기

4.1.6 쿼리 실행 구조

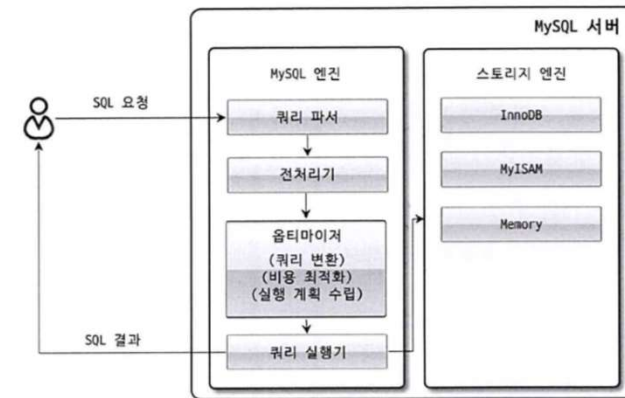


그림 4.6 쿼리 실행 구조

*EXPLAIN*은 예상, *ANALYZE*는 실제다

통계 기반 예상과 실제 데이터의 차이를 비교해야 계획을 검증할 수 있다.

EXPLAIN

- 계획과 추정치를 본다
- *type / key*
- *rows / filtered*
- *Extra*

EXPLAIN ANALYZE

- 실제 실행 결과를 측정한다
- *actual time*
- *rows / loops*
- 추정 오차

추정값이어도 실행 계획을 고르는 근거다

EXPLAIN은 시간을 재는 도구가 아니라 옵티마이저의 판단을 설명한다.

실행 전 위험 발견

*type=ALL · rows=5,000,000*이면 실제 실행 없이도 읽기 폭을 의심한다.

선택 이유 확인

*possible_keys=idx_status*여도 *key=NULL*이면 전체 스캔 비용이 더 싸다고 본 것이다.

변경 전후 비교

*ALL → ref, rows 1,000,000 → 30*처럼 접근 경로가 바뀌었는지 확인한다.

추정 오류도 단서

*rows=100*인데 *actual rows=100,000*이면 통계·편향·상관 관계를 의심한다.

EXPLAIN으로 가설을 세우고 ANALYZE로 검증한다

예상 <i>rows=100</i>	1 · EXPLAIN	<i>type · key · rows · filtered · Extra</i> 로 예상 경로를 읽는다.
실제 <i>actual rows=100,000</i>	2 · ANALYZE	<i>actual time · actual rows · loops</i> 로 실제 작업량을 확인한다.
오차 <i>1,000배</i>	3 · 차이 해석	<i>rows=100 → actual rows=100,000</i> 이면 선택도 추정이 <i>1,000배</i> 틀렸다.
	4 · 개선	통계 갱신 · 히스토그램 · 인덱스 · 쿼리를 바꾸고 다시 비교한다.

이 차이가 개선 작업을 결정한다.

*EXPLAIN*은 네 가지 질문으로 읽는다

많은 컬럼을 외우기보다, 계획이 만든
이야기를 순서대로 확인한다.

1 · 어디를

*table*과 접근 순서

조인에서 먼저 읽는 테이블은?

2 · 어떻게

*type*과 *key*

전체인가, 범위인가, *lookup*인가?

3 · 얼마나

rows × *filtered*

다음 연산에 몇 행이 넘어가는가?

4 · 무엇을 더

Extra

필터·정렬·임시 테이블·커버링은?

ALL은 테이블 전체 스캔이다

언제 나오나

- 사용할 시작점이 없음
- 조건이 인덱스를 못 씀
- 대부분 행이 필요해 더 저렴함

이 화면

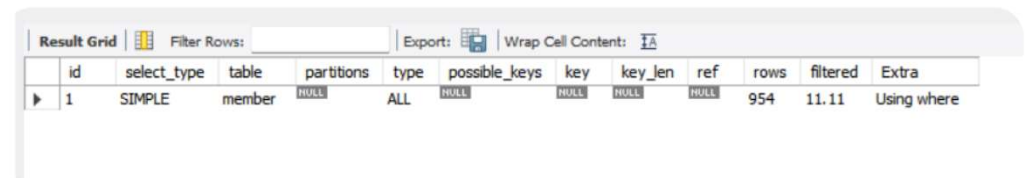
type=ALL · key=NULL

rows=954 · filtered=11.11

Extra=Using where

판단

ALL 자체보다 테이블 크기와 실제 읽기량을 본다.



The screenshot shows a MySQL EXPLAIN output for a query. The 'type' column is 'ALL', indicating a full table scan. The 'key' column is 'NULL', indicating no index is used. The 'rows' column is 954, and the 'filtered' column is 11.11. The 'Extra' column is 'Using where'.

	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	member	NULL	ALL	NULL	NULL	NULL	NULL	954	11.11	Using where

*index*는 인덱스 전체 스캔이다

*ALL*과 공통점

처음부터 끝까지 전체를 읽는다.

다른 점

테이블이 아니라 더 작은 인덱스 리프를 읽을 수 있다.

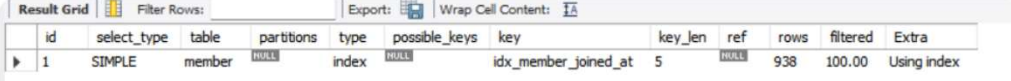
이 화면

*possible_keys=NULL*인데 *key*가 선택됨

COUNT()*를 인덱스만으로 처리해 *Using index*

면접 한 문장

*index*는 빠른 *lookup*이 아니라 *full index scan*입니다.



The screenshot shows a MySQL query result grid. The table has columns: id, select_type, table, partitions, type, possible_keys, key, key_len, ref, rows, filtered, and Extra. The first row shows a full index scan for a COUNT(*) query on the 'member' table.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	member	NULL	index	NULL	idx_member_joined_at	5	NULL	938	100.00	Using index

동등성·유일성·범위가 *type*을 가른다

*type*은 접근 방식이다. 같은 이름도 rows와 반복 횟수까지 함께 판단한다.

const

PK·UNIQUE 전체 키를 상수와 = 비교

쿼리 시작 시 최대 1행

eq_ref

조인에서 선행 행마다 *UNIQUE NOT NULL* 전체 키로 최대 1행

ref

비고유 인덱스 또는 선두 일부를 = 비교

여러 행 가능

range

<, >, *BETWEEN*, *IN*, *LIKE* 'A%'처럼 시작·끝 범위를 만듦

순위	TYPE 값	성능 수준	주요 특징
	1system	최상 (Ideal)	단 0개 또는 1개의 레코드 접근 (매우 드뭄)
	2const	최상 (Ideal)	PK 또는 UNIQUE 키로 1개의 행에 상수처럼 즉시 접근
✓	3eq_ref	최상 (Ideal)	조인에서 PK/UNIQUE 키로 정확히 1개의 행 접근
✓	4ref	매우 좋음 (Excellent)	일반 인덱스로 일치하는 여러 행 접근
	5fulltext	좋음 (Good)	FULLTEXT 인덱스 검색
	6ref_or_null	양호 (Good)	ref + NULL 값 포함 검색
	7index_merge	양호 (Good)	여러 인덱스 검색 결과를 병합하여 사용
	8unique_subquery	양호 (Good)	IN 절 내 UNIQUE 서브쿼리 최적화
	9index_subquery	양호 (Good)	IN 절 내 일반 서브쿼리 처리
✓	10range	주의 필요 (Fair)	인덱스를 사용한 범위 검색 (<, >, BETWEEN, IN)
	11index	느림 (Poor)	인덱스 전체를 처음부터 끝까지 스캔
	12ALL	최악 (Worst)	테이블 전체를 스캔 (Full Table Scan)

인덱스 선택은 네 컬럼을 연결해서 읽는다

possible_keys

행 탐색 조건에 사용할 수 있었던 후보 인덱스

-> *Where*나 *join*으로 조회 범위를 줄이는데 사용할 수 있는
인덱스 후보

key_len

접근 조건에 사용한 최대 키 길이

복합 인덱스 접두사의 단서

key

비용 비교 끝에 실제로 선택한 인덱스

ref

인덱스와 비교하는 상수 또는 선행 테이블 컬럼

sql



```
CREATE INDEX idx_email ON member(email);
```

```
EXPLAIN
```

```
SELECT email
```

```
FROM member;
```

가능한 결과:

text



```
type          index
possible_keys NULL
key           idx_email
Extra         Using index
```

해석하면:

1. WHERE 조건이 없으므로 검색 범위를 줄일 인덱스 후보가 없다.
→ possible_keys = NULL
2. email 만 조회하므로 테이블 행 전체를 읽지 않고 idx_email 만 읽어도 된다.
→ key = idx_email
3. 인덱스 전체를 읽는다.
→ type = index



*rows*와 *filtered*는 다음 단계의 예상 입력량이다

*rows*는 반환 행 수가 아니라 접근 방식으로 조사할 예상 행 수다.



rows

읽어 볼 것으로 추정한 행 수



filtered

테이블 조건을 통과할 것으로 추정한 비율



$rows \times filtered / 100$

다음 *iterator*로 넘어갈 예상 행 수

*Extra*는 접근 이후 남은 일을 말한다

Using where

읽은 후보에 조건을 추가 적용

Using index

필요한 컬럼을 인덱스만으로 해결 — 커버링

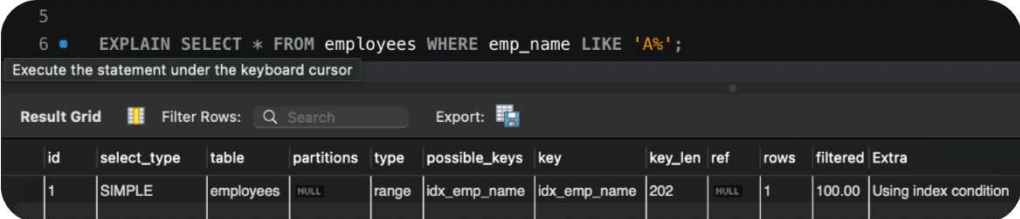
Using index condition

ICP가 인덱스 엔트리를 먼저 검사

통과한 건은 실제 행을 다시 읽음

이 화면

*SELECT ** 때문에 커버링되지 않아 *Using index condition*



The screenshot shows a database query execution plan for the statement: `EXPLAIN SELECT * FROM employees WHERE emp_name LIKE 'A%';`. The plan indicates that the query is using an index condition (ICP) on the `idx_emp_name` index. The 'Extra' column shows 'Using index condition'.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees	HULL	range	idx_emp_name	idx_emp_name	202	HULL	1	100.00	Using index condition

정렬·그룹핑 *Extra*는 추가 작업의 신호다

Using filesort

인덱스 순서만으로 해결하지 못한 별도 정렬
반드시 디스크 정렬은 아님

rows	filtered	Extra
9	100	Using temporary

Using temporary

그룹·중복 제거 등에 내부 임시 테이블 사용
반드시 디스크 테이블은 아님

Using index for group-by

그룹별 일부 인덱스 엔트리만 읽는 *Loose Index Scan*

신호를 봤다면 *rows*와 실제 시간을 확인한다.

*ANALYZE*는 안쪽 *iterator*부터 읽는다

→ *Filter: (emp_no = 10001)*

(cost=10085 rows=10003)

(actual time=24.8..24.8 rows=1 loops=1)

→ *Table scan on employees*

(cost=10085 rows=100000)

(actual time=0.08..21.5 rows=100000 loops=1)

읽는 순서

1. 가장 깊은 자식 연산
2. 부모의 필터·조인
3. 최종 결과

actual time=a..b

첫 행까지 / 모든 행까지의 시간

rows × loops

반복당 행 수와 반복 횟수를 함께 본다.

B+Tree는 시작점을 찾고 정렬된 리프를 읽는다

높은 *fan-out*

한 페이지가 많은 자식 페이지를 가리켜 트리가 얇다.

내부 노드

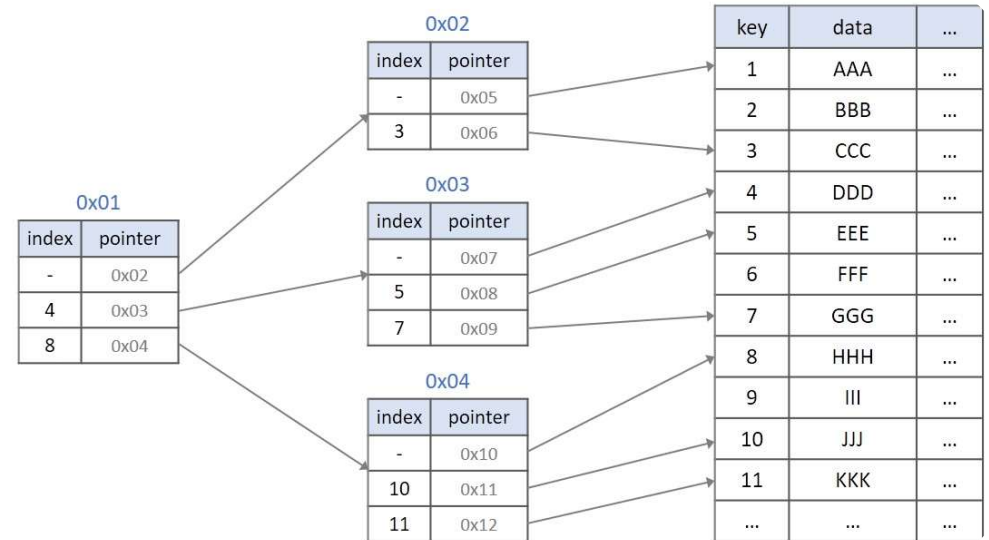
키와 자식 페이지 포인터로 탐색 방향을 정한다.

리프 노드

키 순서로 연결되어 동등·범위·정렬을 지원한다.

트레이드오프

인덱스가 늘면 쓰기와 저장 비용도 늘어난다.



Range Scan은 seek, scan, row lookup으로 나뉜다

1 · seek

루트에서 조건의 시작 리프까지 내려간다.

2 · scan

종료 조건까지 정렬된 리프를 순서대로 읽는다.

3 · row lookup

커버링이 아니면 각 PK로 클러스터드 인덱스를 다시 찾는다.

면접 한 문장

range라고 항상 빠른 것이 아니라 범위 크기와 row lookup

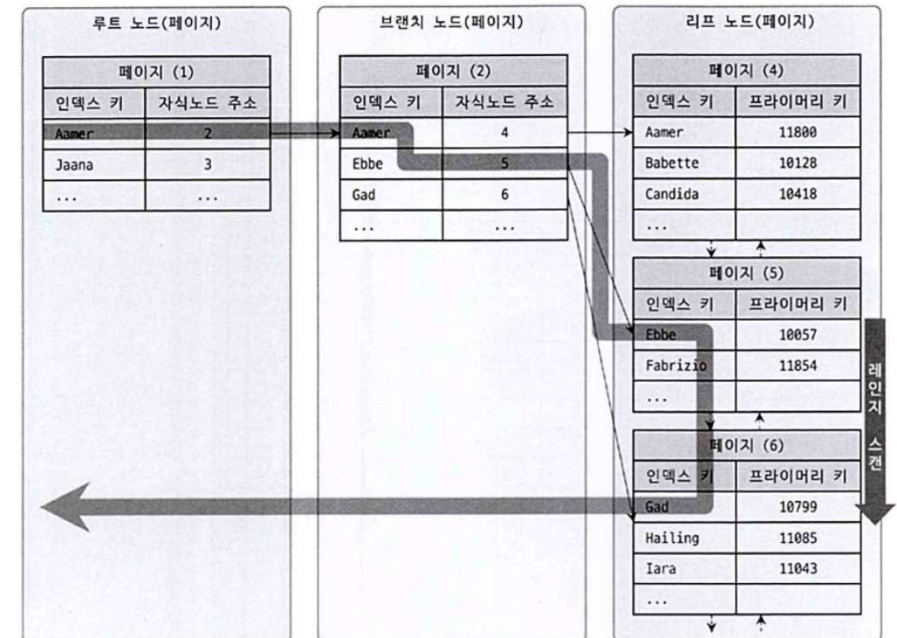


그림 8.8 인덱스를 이용한 레인지 스캔

복합 인덱스는 왼쪽부터 연속 범위를 만든다

*INDEX(region, age, joined_at)*는 사전식으로 정렬된다.

Equality prefix

*region='SEOUL'*처럼 앞 컬럼을 고정하면 다음 컬럼 순서를 활용한다.

First range

age>20 뒤 *joined_at*은 탐색 범위를 더 줄이는 데 제한이 있다.

Covering / ICP

범위에 못 써도 인덱스 필터나 결과 반환에는 도움을 줄 수 있다.

SARGable

*LIKE 'Kim%'*는 범위 가능. *LIKE '%Kim'·YEAR(column)*는 불리하다.

옵티마이저는 인덱스 유무가 아니라 총비용을 비교한다

선택도가 낮거나 결과가 많으면 인덱스가 있어도 ALL이 더 쌀 수 있다.



읽기 범위

예상 *rows*와 데이터 분포

작은 테이블은 전체 스캔도 합리적



row lookup

세컨더리 인덱스에서 찾은 *PK*마다
실제 행을 다시 읽는 비용



후속 작업

filesort·temporary·조인 loops·커
버링 여부를 모두 합산

인덱스 순서가 맞으면 정렬 없이 흘러보낸다

ORDER BY age, joined_at

INDEX(age, joined_at) 순서와 일치

이 화면

type=index · rows=10

key=idx_member_age_joined_at

Extra=Using index

Using filesort 없음

*LIMIT*의 효과

앞 10개를 읽으면 조기에 멈출 수 있다.



	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	member	<i>NULL</i>	<i>index</i>	<i>NULL</i>	idx_member_age_joined_at	9	<i>NULL</i>	10	100.00	Using index

Filesort는 메모리 여부가 아니라 별도 정렬 단계다

Single-pass

정렬 키와 *SELECT* 데이터를 소트 버퍼에 함께 담고 정렬한다.

Two-pass

정렬 키와 행 식별자를 정렬한 뒤 결과 행을 다시 읽는다.

선택 기준

행 크기·버퍼 크기·반환 컬럼에 따라 달라진다.

오해 금지

*Using filesort*의 *file*은 디스크 사용을 보장하지 않는다.

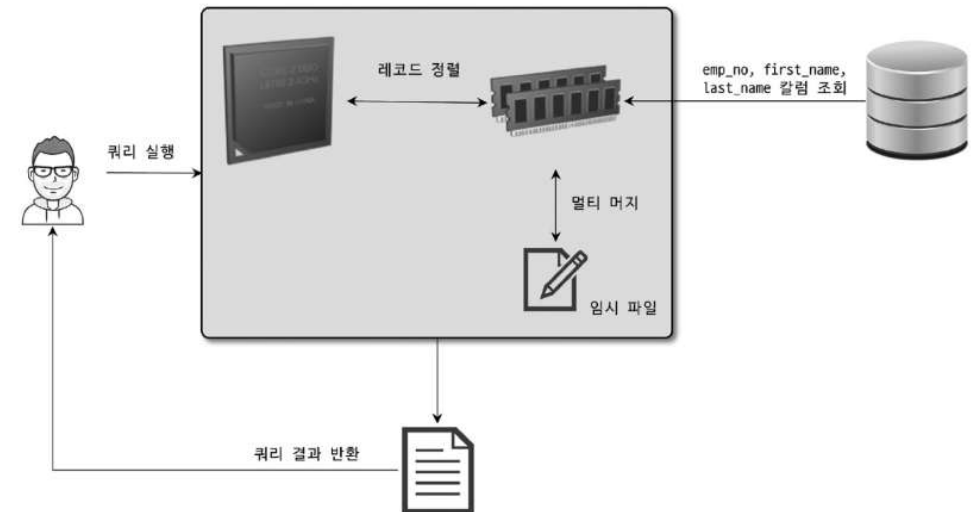


그림 9.2 싱글 패스(Single-pass) 정렬 방식

조인에서는 반복 횟수와 정렬 위치가 비용을 만든다

드라이빙 테이블 m

$type=ALL \cdot Using filesort$

먼저 읽고 정렬한다.



select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
SIMPLE	m	10000	ALL	PRIMARY				938	100.00	Using filesort
SIMPLE	r	10000	ref	idx_reservation_member_id,idx_reservation_m...	idx_reservation_member_id	8	optimizer_lab.m.id	1	100.00	Using index

드라이브 테이블 r

$type=ref \cdot ref=m.id$

선행 행마다 인덱스 *lookup*한다.

비용 감각

선행 결과 행 수 $\times$ 드라이브 *lookup* 비용

FROM 순서가 아니라 옵티마이저의 비용 판단으로 순서가

GROUP BY 전에 *WHERE*로 입력 행을 줄인다

논리 순서를 알면 조건을 어디에 두어야 하는지 설명할 수 있다.

WHERE

그룹을 만들기 전에 개별 행을 제거한다.

GROUP BY

남은 행을 키별로 모아 집계한다.

HAVING

COUNT() > 10*처럼 집계 결과에 조건을 적용한다.

ORDER BY

그룹 결과의 순서가 필요하면 명시한다.

*Tight*는 모두 읽고, *Loose*는 그룹 경계를 건너뛴다

둘 다 인덱스 순서를 활용하지만 읽는 엔트리 수가 다르다.

Tight Index Scan

- 조건 범위의 인덱스 엔트리를 모두 읽고 그룹 경계를 계산
- *COUNT·SUM* 등
- 전용 *Extra* 없음
- 커버링이면 *Using index* 가능

Loose Index Scan

- 그룹별 첫 값 또는 마지막 값만 읽고 다음 그룹으로 점프
- *GROUP BY*가 왼쪽 접두사
- 대표적으로 *MIN/MAX*
- *Using index for group-by*

*DISTINCT*도 중복을 모으는 비용을 가진다

SELECT DISTINCT a, b

(a,b) 조합 전체의 중복을 제거한다.

COUNT(DISTINCT region)

집계 내부에서 고유값 상태가 필요하다.

이 화면

Using index for group-by (scanning)

그룹 간 점프보다 인덱스 전체 순차 스캔이 싸다고 선택

*Using index*와 다름

일반 커버링 표시가 아니라 *GROUP BY/DISTINCT* 최적화

key	key_len	ref	rows	filtered	Extra
region_idx_order_distinct_region	82	<null>	8	100	Using index for group-by (scanning)

InnoDB 보조 인덱스는 PK를 통해 실제 행을 찾는다

클러스터드 인덱스

PK 순서의 리프에 행 전체를 저장한다.



그림 8.5 B-Tree의 리프 노드와 테이블 데이터 레코드(MyISAM)

세컨더리 인덱스

세컨더리 키 + PK를 저장한다.

비커버링 조회

secondary B+Tree → PK 획득 → clustered B+Tree 재 탐색



그림 8.6 B-Tree의 리프 노드와 테이블 데이터 레코드(InnoDB)

설계 영향

긴 PK는 모든 세컨더리 인덱스를 함께 크게 만든다.

커버링은 행 접근을 없애고 ICP는 후보만 줄인다

Using index · 커버링

조건·조인·정렬·반환 컬럼이 모두 한 인덱스에 있어 클러스터드 행을 다시 읽지 않는다.

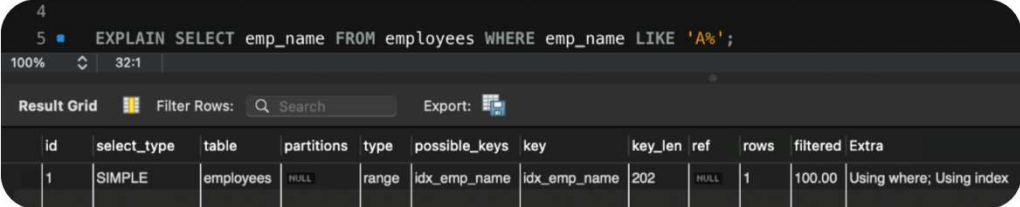
Using index condition · ICP

인덱스 엔트리에서 조건을 먼저 검사하지만 통과한 건은 실제 행을 읽는다.

이 화면

*emp_name*만 반환하므로 *idx_emp_name*으로 커버링

Extra=Using where; Using index



The screenshot shows a database query execution plan for the query: `EXPLAIN SELECT emp_name FROM employees WHERE emp_name LIKE 'A%';`. The plan indicates that the `idx_emp_name` index is used, and the `Extra` column shows `Using where; Using index`, which is a covering index scan.

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	employees		range	idx_emp_name	idx_emp_name	202		1	100.00	Using where; Using index

하나의 쿼리에 다섯 주차가 모두 들어 있다

```
SELECT region, COUNT(*) AS cnt
FROM member
WHERE status = 'ACTIVE'
      AND age BETWEEN 20 AND 29
GROUP BY region
ORDER BY cnt DESC
LIMIT 10;
```

1 · *type*과 *key*로 접근 경로 확인

2 · (*status*, *age*, *region*) 왼쪽 접두사 점검

3 · *rows* × *filtered*로 집계 입력량 추정

4 · *Extra*에서 *temporary/filesort* 확인

5 · *SELECT* 컬럼과 *PK lookup*으로 커버링 가능성 확인

마지막은 *EXPLAIN ANALYZE*로 실제 *rows·loops·time* 검증

드문 *type*은 이름보다 접근 원리를 기억한다

system

사실상 1행인 *const*의 특수 사례

ref_or_null

동등 *lookup*과 *NULL* 행을 함께 탐색

index_merge

여러 인덱스 범위 결과를 병합

subquery 계열

unique_subquery·*index_subquery*는 *IN* 서브쿼리 전용 접근

Extra 네 문장을 구분하면 실행 계획이 선명해진다

면접에서는 용어를 번역하지 말고 제거되는 비용을 말한다.

Using index

커버링 — 기본 테이블의 추가 행 조회를 없앤다.

Using index
condition

ICP — 행 조회 전에 인덱스 조건으로 후보를 줄인다.

Using filesort

별도 정렬 — 디스크 사용 여부는 별도 확인한다.

Using temporary

내부 임시 테이블 — 중간 결과 크기와 실제 시간을 본다.

면접에서는 실행 계획을 다섯 문장으로 답한다

좋고 나쁨을 말하기 전에 계획이 한 일을 설명한다

답변 순서는 접근 경로 → 읽기량 → 인덱스 선택 → 추가 작업 → 실제 검증이다.

- ① 1 · *type*으로 전체·범위·*lookup*을 구분합니다
- ② 2 · *key*와 *key_len*으로 사용한 접두사를 봅니다
- ③ 3 · *rows* × *filtered*와 *loops*로 작업량을 봅니다
- ④ 4 · *Extra*로 필터·정렬·임시 테이블·커버링을 봅니다
- ⑤ 5 · *EXPLAIN ANALYZE*로 실제 *rows*와 시간을 검증합니다